OptiCrop: AI-Powered Crop Recommendation System

Project Description:

OptiCrop is a machinelearning powered decision-support system that helps farmers and agricultural planners choose the crop best suited to a given plot of land. Instead of relying on tradition or guesswork, the platform takes seven measurable inputs — Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall — and returns the single crop most likely to thrive under those conditions. The system is built around a Caption-style single-response design philosophy: one submission from the user produces one clear, actionable recommendation, removing the need to manually cross-reference agronomy charts or consult multiple sources.

Utilizing a Scikit-Learn classification model trained on a labelled crop recommendation dataset, OptiCrop processes soil and climate readings to deliver an instant, data-driven recommendation. Built with Flask on the backend and a lightweight HTML, CSS, and JavaScript frontend, the platform ensures a fast, form-based, user-friendly experience. With server-side input validation and a serialized model loaded via Pickle, OptiCrop empowers farmers, agricultural extension workers, and agri-tech businesses to make informed planting decisions with confidence.

Scenarios

Scenario 1:A smallholder farmer in a region with high rainfall and moderate temperature tests their soil and finds high nitrogen and potassium levels but low phosphorus. After entering the seven readings into OptiCrop, the system analyses the combination and recommends rice as the most suitable crop, along with the confidence level of the prediction.

Scenario 2: An agricultural extension officer is advising several farmers in a semi-arid zone with low rainfall and high temperature. By entering each farmer's soil test values one at a time, the officer uses OptiCrop to quickly compare recommendations across plots, receiving suggestions such as millet or cotton depending on the specific nutrient balance of each field.

Scenario 3: A commercial farm manager is planning the next planting cycle and wants to validate an existing crop choice. Entering the farm's latest soil nutrient and weather readings into OptiCrop confirms that the currently planned crop matches the model's recommendation, giving the manager confidence to proceed, or alerts them to a better-suited alternative crop if the values do not align.

Technical Architecture:

Description: OptiCropfollows a modular three-tier architecture designed to deliver a fast, reliable crop recommendation experience. At the center of the architecture is the Flask-based backend, which receives soil and environmental readings from the frontend, forwards them to the trained Machine Learning module, and returns the prediction to the user. The frontend, built with HTML, CSS, and

JavaScript, provides a simple form-based interface for entering the seven input parameters. The frontend submits user input to the backend, which validates the request before passing it to the Machine Learning module. The Machine Learning module — a Scikit-Learn classifier deserialized from a Pickle file at application startup — generates the crop prediction and returns it to the backend, which then formats the result and sends the recommended crop name back to the frontend for display. Because OptiCrop does not depend on any external generative AI service, the entire prediction pipeline runs locally, ensuring both speed and data privacy.

(Note: OptiCrop does not use a traditional database. All application logic revolves around the trained model file and does not require persistent storage of user records; this is elaborated further in the Database section below.)

Pre-requisites:

- Flask Framework Knowledge: Flask Documentation
- Scikit-Learn Familiarity: Scikit-Learn Documentation
- HTML, CSS, and JavaScript Skills: W3Schools HTML/CSS/JavaScript Tutorials
- Python Programming Proficiency: Python Documentation
- Pandas and NumPy for Data Handling: Pandas Documentation, NumPy Documentation
- Version Control with Git: Git Documentation
- Development Environment Setup: Flask Installation Guide

Project Workflow:

Milestone 1: Data Collection and Model Selection

- Activity 1.1: Acquire and understand the crop recommendation dataset, reviewing its features and target labels.
- Activity 1.2: Research and select the appropriate Scikit-Learn algorithm for multi-class crop classification.
- Activity 1.3: Define the architecture of the application, detailing interactions between the frontend, backend, and ML model.
- Activity 1.4: Set up the development environment, installing the necessary libraries and dependencies for Flask, Pandas, NumPy, and Scikit-Learn.

Milestone 2: Data Preprocessing and Model Training

- Activity 2.1: Clean and preprocess the dataset using Pandas, checking for missing values and verifying feature ranges.
- Activity 2.2: Split the dataset into training and testing sets and train the classification model using Scikit-Learn.
- Activity 2.3: Evaluate model accuracy and serialize the trained model using Pickle for use in the Flask application.

Milestone 3: app.py Development

- Activity 3.1: Write the main application logic in app.py, establishing routes that load the serialized model and serve predictions.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a clean and intuitive input form.
- Activity 4.2: Create dynamic templates with Flask's render_template to display the recommended crop based on user input.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment by configuring the server environment and verifying all dependencies.
- Activity 5.2: Test the application end-to-end and, where required, deploy it to a hosting platform for public access.

Milestone 6: Conclusion

Milestone 1: Data Collection and Model Selection

Inthismilestone,thefocusison understanding thecroprecommendation dataset and selecting the appropriate Scikit-Learn algorithm for OptiCrop's classification needs. This involves reviewing the dataset's structure, researching suitable models, and preparing the development environment before any code is written.

Activity 1.1: Understand the Dataset

The Crop Recommendation Dataset contains rows of soil and climate readings, each labelled with the crop that is best suited to those conditions. Before training begins, the dataset is inspected for the range and distribution of each feature.

- Nitrogen (N), Phosphorus (P), Potassium (K): numeric soil nutrient values expressed in standard units.
- Temperature and Humidity: average environmental readings for the plot.
- pH: the acidity or alkalinity of the soil.
- Rainfall: average rainfall for the region, expressed in millimetres.
- Label: the target column identifying the recommended crop, such as rice, maize, chickpea, or cotton.

Activity 1.2: Research and Select the Appropriate Algorithm

Here are the things that needed to be researched to select the best model for the project:

- Understand the Project Requirements: Review the specific needs of the OptiCrop application, focusing on multi-class classification across a large number of possible crop labels.
- Explore Scikit-Learn's Model Documentation: Examine the available classifiers, including their assumptions, training speed, and suitability for tabular agricultural data.
- Evaluate Model Performance: Compare candidate models based on classification accuracy, resistance to overfitting, and interpretability.
- Select the Optimal Model: Choose a Random Forest Classifier for its strong accuracy on tabular data, its resistance to overfitting through ensemble averaging, and its ability to handle non-linear relationships between soil nutrients and crop suitability.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Overview: Identify the components of the application, including the frontend (HTML, CSS, JavaScript), the Flask backend, and the Machine Learning module.

- Detail Frontend Functionality: Outline how users interact with the application, entering the seven soil and environmental readings through a form.
- Outline Backend Responsibilities: Specify how the Flask backend handles incoming POST requests to /predict, validates numeric input, and passes the values to the trained model.
- Describe Model Integration Points: Define how the backend loads the serialized model at startup using Pickle, constructs a feature array from user input, calls model.predict(), and returns the resulting crop name to the frontend.

Activity 1.4: Set Up the Development Environment

- Install Python and Pip: Ensure Python 3.8+ is installed along with pip for dependency management.
- Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

- Install Flask: Use pip to install Flask.

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- Install Scikit-Learn, Pandas, and NumPy: Install the core data science libraries used for training.

```
(myenv) D:\projects> pip install scikit-learn==1.4.2 pandas==2.2.2 numpy==1.26.4
```

- Set Up Application Structure: Create the project directory structure including folders for the dataset, the trained model, static files, and templates.

```
dataset/
model/
notebook/
static/
  css/
  js/
templates/
app.py
train_model.py
requirements.txt
```

Milestone 2: Data Preprocessing and Model Training

Milestone 2 covers the preparationof the dataset and thetrainingoftheclassification model that powers OptiCrop's recommendations. This milestone establishes train_model.py as the script responsible for turning raw data into a deployable, serialized model.

Activity 2.1: Preprocess the Dataset

- Load the Dataset: Read the crop recommendation CSV file into a Pandas DataFrame.
- Check Data Quality: Verify there are no missing values and confirm that each numeric feature falls within a realistic agronomic range.
- Separate Features and Target: Split the DataFrame into an input feature matrix (N, P, K, temperature, humidity, pH, rainfall) and the target label column.

```
import pandas as pd

df = pd.read_csv('dataset/crop_recommendation.csv')

features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
X = df[features]
y = df['label']
```

Activity 2.2: Train the Classification Model

- Split the Data: Divide the dataset into training and testing sets, typically using an 80/20 split.
- Train the Model: Fit a Random Forest Classifier on the training data.
- Tune Hyperparameters: Adjust parameters such as n_estimators and max_depth to balance accuracy and generalization.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(
    n_estimators=200, max_depth=None, random_state=42
)
model.fit(X_train, y_train)
```

Activity 2.3: Evaluate and Serialize the Model

- Evaluate Accuracy: Measure the model's performance on the held-out test set using accuracy score and a classification report.

- Serialize the Model: Save the trained model to disk using Pickle so that it can be loaded instantly by the Flask application without retraining.

```
import pickle
from sklearn.metrics import accuracy_score, classification_report

predictions = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions))

with open('model/crop_model.pkl', 'wb') as f:
    pickle.dump(model, f)
```

Once train_model.py has been executed, the resulting crop_model.pkl file becomes the single artifact that the Flask backend depends on for inference, decoupling model training from the runtime application.

Milestone 3: app.py Development

Milestone3 focusesoncreating thecore logicof app.py, which serves as the backbone of the OptiCrop application. In this milestone, the serialized model is loaded once at startup, a prediction route is defined, and input validation is applied to ensure reliable behaviour.

Activity 3.1: Writing the Main Application Logic in app.py

- Load the Trained Model: At application startup, deserialize crop_model.pkl using Pickle so it is ready to serve predictions without delay.
- Define the Home Route: / — renders the main index.html interface containing the input form.

```
from flask import Flask, request, render_template
import pickle
import numpy as np

app = Flask(__name__)

with open('model/crop_model.pkl', 'rb') as f:
    model = pickle.load(f)

@app.route('/')
def index():
    return render_template('index.html')
```

- Define the Prediction Route: /predict — a POST endpoint that reads the seven form values, validates them, and returns the recommended crop.

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        n = float(request.form['nitrogen'])
        p = float(request.form['phosphorus'])
        k = float(request.form['potassium'])
        temperature = float(request.form['temperature'])
        humidity = float(request.form['humidity'])
        ph = float(request.form['ph'])
        rainfall = float(request.form['rainfall'])
    except (KeyError, ValueError):
        return render_template(
            'index.html',
            error='Please enter valid numeric values for all fields.'
        )

    features = np.array([[n, p, k, temperature, humidity, ph, rainfall]])
    prediction = model.predict(features)[0]

    return render_template('index.html', prediction=prediction)

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

- Apply Input Validation: Return a friendly error message on the same page if any field is missing or non-numeric, rather than allowing the application to crash.
- Route Form Submissions: The form in index.html submits directly to the /predict endpoint, which processes the data and re-renders the page with the result.

Milestone 4: Frontend Development

Milestone 4 focuses ondevelopinga clean, accessible interface for OptiCrop. This involves designing a simple, form-driven layout with responsive HTML and CSS, and using Flask's template rendering to display the recommended crop once the model has produced a prediction.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the OptiCrop name and tagline, an input form, and a results section.
- Use semantic HTML elements and clearly labelled fields so the interface remains accessible to users with varying levels of technical familiarity.
- Integrate Google Fonts for clean typography and simple iconography to label each input field (nutrient values, temperature, humidity, pH, and rainfall).

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a clean, earth-toned agricultural theme suited to the subject matter.
- Use flexbox to organize the input form into a two-column grid of labelled fields, with the submit button spanning the full width beneath it.
- Add media queries so the layout adapts across desktop, tablet, and mobile screen sizes.

Create Clear Feedback for the Result:

- Result Card: a single highlighted card that displays the recommended crop name prominently once a prediction has been made.
- Error Banner: a clearly styled message that appears above the form if any input is missing or invalid, guiding the user to correct it.

Activity 4.2: Rendering Results with Flask Templates

- Handle Form Submission: The form uses the POST method and submits directly to /predict; no client-side JavaScript framework is required for the core prediction flow.
- Render Results Conditionally: Using Jinja2 templating in index.html, the page checks whether a prediction or an error value has been passed from the backend and displays the corresponding section.

```
{% if prediction %}
<div class="result-card">
  <h2>Recommended Crop</h2>
  <p class="crop-name">{{ prediction }}</p>
```

```
</div>
{% elif error %}
<div class="error-banner">{{ error }}</div>
{% endif %}
```

- Preserve Form State: Retain the user's last submitted values in the form fields after a prediction or error, so they do not need to re-enter all seven readings.

Milestone 5: Deployment

InMilestone 5, thefocus ison deploying the OptiCrop application first locally to verify correct operation, and then to a hosting platform if public access is required. This involves configuring the server environment, managing dependencies, and testing the full prediction pipeline end-to-end.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

- Confirm the Model File Is Present: Ensure crop_model.pkl exists inside the model/ directory before starting the server, since app.py loads it at startup.

Activity 5.2: Local Testing and Verification

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5000" to access the application.
- Test the prediction form using several different combinations of soil nutrient and climate values to confirm that varied, sensible crop recommendations are returned.
- Test the validation logic by submitting empty or non-numeric values to confirm that the error banner displays correctly.

Activity 5.3: Optional Public Deployment

Where public access is required, the Flask application can be deployed to a cloud platform such as Render, PythonAnywhere, or Heroku. The deployment process involves committing the project — including the trained model file, requirements.txt, and a Procfile or equivalent start command — to a Git repository, then connecting that repository to the chosen hosting platform. Because OptiCrop's model is loaded from a local Pickle file rather than an external API, no additional API keys or secrets are required for deployment, simplifying the configuration process considerably compared to systems that depend on third-party AI services.

Exploring the Website's Web Pages:

Home / Prediction Page:

Description: The single-page interface of OptiCrop serves as both the landing page and the prediction tool. It features a clean, agricultural-themed layout with a header displaying the OptiCrop name and a short tagline describing the platform's purpose. The main section contains the seven-field input form and, once a prediction has been made, the results panel — all on one seamless page.

Input Section:

Description: Aclearly labelled form containing seven numeric input fields — Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall — arranged in a two-column grid. Each field includes a short placeholder indicating the expected unit or typical range. A single "Recommend Crop" button submits the form to the backend for processing.

Results Section:

Description:Revealed after a successful submission, this section displays the recommended crop name inside a prominently styled result card. If the submitted values are invalid or incomplete, an error banner appears above the form instead, guiding the user to correct their input before resubmitting.

Conclusion:

OptiCrop is a machine learning platform built with Flask and Scikit-Learn that streamlines crop selection for farmers and agricultural planners. It uses a Random Forest Classifier trained on soil nutrient and climate data to instantly recommend the most suitable crop based on seven measurable inputs. The project, which included dataset preparation, model training and serialization, backend development, and frontend design, highlights how a focused machine learning model and a simple, form-driven interface can meaningfully support agricultural decision-making. Looking ahead, OptiCrop offers significant opportunities for expansion, such as integrating live weather API data, incorporating IoT soil sensor readings, adding fertilizer and yield prediction modules, and supporting regional languages for wider accessibility. By continuously refining the underlying model and extending the platform's feature set, OptiCrop is well-positioned to evolve from a single-prediction tool into a comprehensive agricultural decision-support assistant for farmers of all scales.